ECE 2564
Fall 2025
Homework 4

Please sign

"I have neither given nor received unauthorized assistance on this assignment. I have not copied any part of this submission from unauthorized resources".

To complete this homework, you need to complete the following steps:

1. Follow the instructions here to create your homework repository. For the homework link, use the below link:
   https://classroom.github.com/a/OFMSruDs

2. Follow the instructions here to import your homework repository to CCS.

3. There are two sections for this homework. Follow the instructions in this document to complete each problem. As you follow the instructions, you are both developing code in CCS and creating an answer/solution document. The solution document could be this word document where you have entered your answers, screenshots, etc.

4. Once you have finished each problem, follow the instructions here to submit your work to GitHub. We hope that by this homework, you have learned how to submit your work to GitHub. This assignment does not ask you to send the screenshots of your code. Your code will be solely graded based on your GitHub submission. Please allow enough time before the deadline for GitHub submission and double-checking your code. If you have any issue with GitHub and the deadline is close, zip-up your HW4 folder and submit it in a help ticket. No late assignment is accepted due to technical difficulties with GitHub.

5. Turn your solution document (perhaps a modified version of this document) to Canvas. Your grade will depend on both these submissions.

6. Sign the Honor Code statement.

In this homework, you will work with buttons and LEDs on the Launchpad and BoosterPack. There are a total of 4 buttons and 7 individual LEDs on these boards with long and sometimes duplicate names. We will use our own naming convention in this course. In our convention, the first letter indicates the board initial, L for Launchpad and B for BoosterPack; The second letter indicates the type of component, B for Button, and L for LED. The last character provides more distinctive information. For example, LB1 means Launchpad Button 1. The below table provides the complete list of buttons and LEDs we will use.

Board	**Name on the user guide**		**Name in this course**	**More info**
Launchpad	S1 (BUTTON1)		LB1	On the left
	S2 (BUTTON2)		LB2	On the right
	LED1		LL1	On the left, red
	LED2_RED (RGBLED_RED)	Launch pad RGB LEDs	LLR	On the right, red
	LED2_GREEN (RGBLED_GREEN)		LLG	On the right, green
	LED2_BLUE (RGBLED_BLUE)		LLB	On the right, blue
BoosterPack	S1 (BUTTON1)		BB1	Top
	S2 (BUTTON2)		BB2	Bottom
	RED_LED	Booster Pack RGB LEDs	BLR	Red
	GRN_LED		BLG	Green
	BLU_LED		BLB	Blue

Problem 0. Enter your GitHub Repository link here.

(40 points) Problem 1.

A. (2 points) Compile and press the debug for S25_HW4_P1 project. Once this code is actively running on the kit, pressing LB2 turns on LL1 and releasing LB2 turns LL1 off. This is the same behavior as the example code we covered in class. Step into `initialize()` function. Make sure your "variables/expressions/registers" view is open. If not, press on view and click on either of "variables", "expressions", or "registers". They will open the same window. Click on "Registers". Scroll until you see P1 in the Name column. Press on the arrow beside P1. That should open a view that shows all the registers associated with Port 1. We have discussed half of these registers in class. Some of them are out of the scope of this course. Your view should be something similar to Figure 1. Step over several times until you reach line 68. Keep an eye on P1DIR. Stepping over `*P1DIR_pointer |= LL1;` should update the value of P1DIR and make the cell yellow indicating a change. Take a screenshot and attach it to your solution.

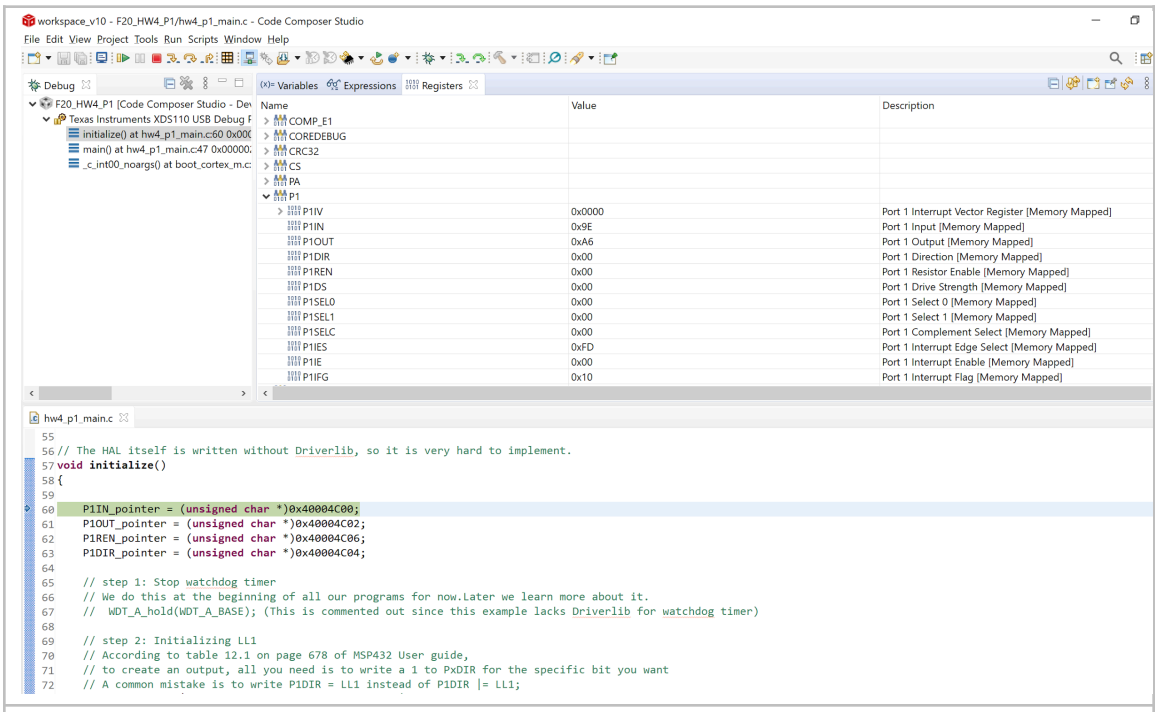


Figure 1 – Registers view of the debugger

B. (4 points) Continue stepping. That will take you back to the main function. Step over several times until you finish one loop of the while(1). Write down the value of P1IN. Then, press your finger on LB2 , the right button of the Launchpad, which is very inconveniently located under BoosterPack. See Figure 2 for how to find this button. With your fingers pressing down on LB2 , step over once. Take a screenshot that shows P1IN is yellow and it has been updated and attach it to your solution. Convert P1IN value before and after this step into binary. Explain in a few sentences if the bit value

corresponding to LB2 matches the Pressed and Released state you expect it to be. In your response, mention user guide pages and certain diagrams if needed.

C. (4 points) Continue pressing on LB2 . Write down the value of P1OUT. Continue stepping until you see P1OUT changes. Take a peek at the kit. You should see LL1 is on. You can release the button now. Convert P1OUT before and after this change to binary. Explain in a few sentences if the bit value corresponding to LL1 matches the on/off state of the LED.

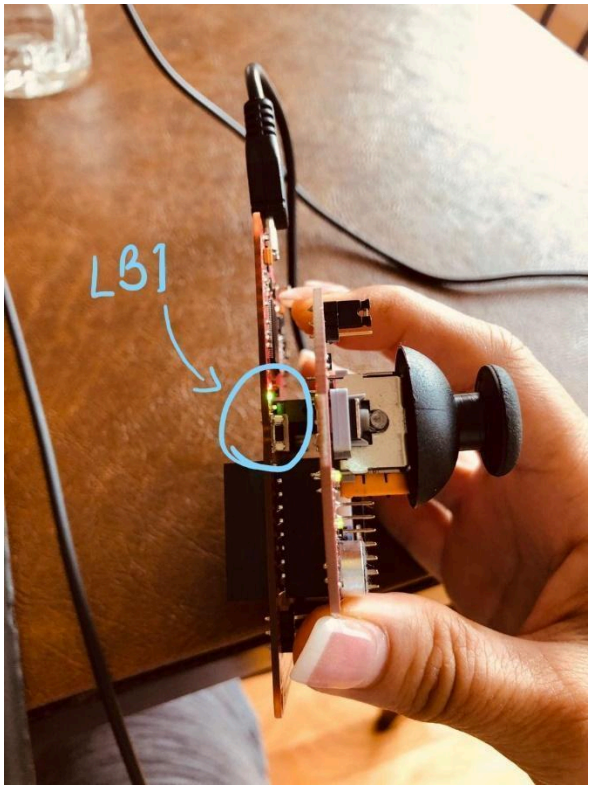


Figure 2 – Location of LB1. Hold the kit sideways with the joystick on the right. LB2 is opposite of LB1

D. (30 points) Add some code in hw4_p1_main.c such that when BB2 is pressed the BLR is on and whenever BB2 is released, BLR is off. Use the same approach as S25_HW4_P1, using memory-mapped addresses and no DriverLib functions. Still, similar to the starter code, use HAL functions to add readability to the code. Note that you will need to look into kit documents to achieve this goal. Take screenshots of the pages from user manual(s) you need and highlight or circle the parts of the document you are using for this part of the homework. Put the highlighted screenshot in your assignment. We highly recommend that you use the debugging techniques you have learned to build your code. Check if BB2 reading is correct and try turning on/off BLR on its own without dependency on BB2. In other words, ***break the problem into smaller pieces and make sure they work flawlessly*** before putting these pieces together. Submit your code to GitHub.

(60 points) Problem 2. In this problem, you will extend the S25_HW4_P2 project, which uses Driverlib functions and builds HAL functions. The goal is to tie the RGB LEDs on BoosterPack to the buttons on BoosterPack such that the functionality in the below table is achieved. For example, if neither BB1 nor BB2 is pressed, the BoosterPack LED light should be magenta (a mixture of various basic RGB LED colors).

BoosterPack buttons pressed	**BoosterPack RGB color**
Neither BB1 nor BB2	magenta
BB1 only	red
BB2 only	blue
BB1 and BB2 together	yellow

Refer to https://sites.google.com/vt.edu/introduction-to-embeddedsystem/digital-io/launchpad-peripherals if you wish to learn about RGB color mixing.

We highly recommend that you try to solve this problem following the below steps:

Step 1: Add the initialization code for configuring the digital i/o pins attached to BB1, BB2, BLR, BLG, and BLB.

Step 2: Write the HAL functions for turning on and off each individual LED on BoosterPack, namely BLR, BLG, and BLB. Debug your code and make sure you can turn these LEDs on and off. You can use LB2 (whose code is already in the starter code) to turn these LEDs on and off and test them. If your code does not work, you might have to go back to step 1 to make sure your initialization is correct.

Step 3: Write the HAL functions for checking if BB1 or BB2 are pressed. Test these buttons with one of your working LEDs. In other words, replace LB2 with BB1 and BB2 one at a time and see if these new buttons work.

Step 4: Write a new function that turns the BoosterPack RGB to red, yellow, blue, or cyan depending on the desired RGB status. The declaration of this function and the enum type enumerating all the states is given to you in the starter code:

```
typedef enum {MAGENTA, RED, BLUE, YELLOW, OFF} color_t;

void Turn_BoosterPack_LED(color_t desiredColor);
```

You can test this part independently of the buttons to make sure it works by calling it several times with various arguments. The speed of the processor is fast, so you should debug the project and step through it instead of just running it. Practice using breakpoints.

Step 5: Update your main such that the desired functionality of this problem (described in the above table) is achieved. Use debugging techniques you have learned if you face any issues. Submit your code to GitHub.